Lekcja 10. Definiowanie klas

Klasa 4 technik programista | Programowanie aplikacji | INF.04

Notatka obejmuje podstawową składnię definicji klasy w C++, specyfikatory dostępu, zmienne członkowskie, funkcje członkowskie, operator zakresu oraz deklarowanie zmiennych obiektowych.

1. Skróty i nowe pojęcia

Pojęcie	Wyjaśnienie
OOP / POO	programowanie zorientowane obiektowo, czyli sposób projektowania programu z użyciem klas i obiektów.
class	słowo kluczowe języka C++ używane do definiowania klasy.
public	specyfikator dostępu oznaczający elementy widoczne również poza klasą.
private	specyfikator dostępu oznaczający elementy widoczne tylko wewnątrz klasy; w class jest domyślny.
protected	specyfikator dostępu ważny przy dziedziczeniu; element dostępny w klasie i klasach pochodnych.
::	operator zakresu; wskazuje, że funkcja lub element należy do konkretnej klasy albo przestrzeni nazw.
field / pole	inna nazwa zmiennej członkowskiej przechowującej dane obiektu.
method / metoda	funkcja członkowska klasy, czyli funkcja należąca do klasy.
constructor / konstruktor	specjalna metoda wywoływana podczas tworzenia obiektu; szczegółowo omawiana dalej.

2. Klasa i jej składniki

Klasa jest złożonym typem danych definiowanym przez programistę. Określa wspólne cechy i zachowania obiektów. Zawiera elementy członkowskie, czyli pola oraz metody.

Obiekt jest instancją klasy. Obiekt powstaje w pamięci i posiada własny zestaw wartości pól. Metody opisane w klasie mogą działać na tych danych.

Schemat pojęciowy:

Klasa opis typu: pola + metody	→	Obiekt konkretna instancja klasy

3. Ogólna składnia klasy

Definicja klasy w języku C++ ma ogólną postać:

```
class nazwa_klasy {
specyfikator_dostepu_1:
definicje_zmiennych_czlonkowskich_1;
deklaracje_funkcji_czlonkowskich_1;
definicje_funkcji_czlonkowskich_1;

specyfikator_dostepu_2:
definicje_zmiennych_czlonkowskich_2;
deklaracje_funkcji_czlonkowskich_2;
```

```
definicje_funkcji_czlonkowskich_2;
};
```

- nazwa_klasy jest identyfikatorem tworzonego typu.
- specyfikator_dostepu określa, skąd można korzystać z elementów klasy.
- Definicja klasy musi zakończyć się średnikiem po nawiasie klamrowym: };

4. Specyfikatory dostępu

Specyfikator dostępu określa widoczność elementów członkowskich klasy.

Specyfikator	Znaczenie	Typowe zastosowanie
public	elementy dostępne z zewnątrz klasy, tam gdzie widoczny jest obiekt	metody stanowiące interfejs klasy
private	elementy dostępne tylko wewnątrz klasy; domyślne w class	pola przechowujące stan obiektu
protected	elementy dostępne w klasie i klasach pochodnych	dziedziczenie, omawiane szerzej później

Jeżeli w definicji klasy nie podamy jawnie specyfikatora dostępu, kompilator przyjmie domyślnie dostęp private.

5. Zmienne członkowskie

Zmienne członkowskie klasy, nazywane także polami lub danymi członkowskimi, reprezentują dane przechowywane w obiektach danej klasy. Ich liczba zależy od potrzeb projektowych. Mogą mieć typy proste, np. int, float, string, albo typy złożone, np. tablice, struktury lub inne klasy.

```
class Uczen {
public:
string imie;
string nazwisko;
string klasa;
};
```

6. Funkcje członkowskie

Funkcje członkowskie klasy, czyli metody, odpowiadają za działania wykonywane na danych przechowywanych w obiekcie. Mogą być zdefiniowane bezpośrednio w klasie albo zadeklarowane w klasie i zdefiniowane poza nią.

Przykład definicji metody bezpośrednio w klasie:

```
class Pracownik {
public:
string imie;
string nazwisko;

void wyswietlDane() {
cout << "Dane pracownika: " << imie << " " << nazwisko << endl;
}
};
```

W tym przykładzie klasa Pracownik ma dwa pola typu string oraz metodę wyswietlDane(). Wszystkie elementy są publiczne, ponieważ umieszczono je po specyfikatorze public.

7. Definiowanie metod poza klasą

Jeżeli w klasie umieszczamy tylko deklarację metody, jej definicję można zapisać poza klasą. Wtedy nazwa metody musi być poprzedzona nazwą klasy i operatorem zakresu ::.

```
class Pracownik {
public:
string imie, nazwisko;
void ustawImie(string);
void ustawNazwisko(string);
void wyswietlDane();
};

void Pracownik::ustawImie(string pImie) {
imie = pImie;
}

void Pracownik::ustawNazwisko(string pNazwisko) {
nazwisko = pNazwisko;
}
```

Zapis Pracownik::ustawImie oznacza, że metoda ustawImie należy do klasy Pracownik.

8. Przykład: klasa Uczen

Na podstawie klasy Pracownik można przygotować klasę Uczen, która przechowuje imię, nazwisko i klasę ucznia oraz udostępnia metody ustawiające i wyświetlające dane.

```
class Uczen {
public:
string imie, nazwisko, klasa;

void ustawImie(string pImie);
void ustawNazwisko(string pNazwisko);
void ustawKlase(string pKlasa);
void wyswietlDane();
void wyswietlPersonalia();
void wyswietlKlase();
};

void Uczen::ustawKlase(string pKlasa) {
klasa = pKlasa;
}

void Uczen::wyswietlPersonalia() {
cout << imie << " " << nazwisko << endl;
}

void Uczen::wyswietlKlase() {
cout << klasa << endl;
}
```

9. Deklarowanie zmiennych obiektowych

Aby w programie korzystać z obiektu, należy go wcześniej utworzyć jako zmienną obiektową. Można to zrobić zaraz po definicji klasy albo później, podobnie jak deklaruje się zwykłe zmienne.

Sposób 1 - deklaracja obiektów po definicji klasy:

```
class Pracownik {
...
} pracownik1, pracownik2;
```

Sposób 2 - deklaracja obiektu po wcześniejszej definicji klasy:

```
Pracownik pracownik;
Uczen uczen1, uczen2;
```

W dalszych lekcjach pojawi się pojęcie konstruktora. Konstruktor jest specjalną metodą, która może zostać wywołana podczas tworzenia obiektu.

10. Do zapamiętania

- Klasa jest złożonym typem danych definiowanym przez programistę.
- Obiekt jest instancją klasy i ma własny stan.
- Pola przechowują dane obiektu, a metody wykonują operacje.
- Specyfikatory dostępu sterują widocznością elementów klasy.
- Operator zakresu :: pozwala definiować metody poza klasą.